代码整合

分治与递归

排列问题：

```
template <class Type>
void Perm (Type list [], int k, int m) {    // 产生list[k:m]的所有排列
    if (k == m) {                           // 只剩下一个元素
        for (int i = 0; i <= m; i++)
            cout << list[i];
        cout << endl;
    }
    else {
        for(int i = k; i <= m; i++) {       // 还有多个元素待排列，递归产生排列
            Swap( list[k], list[i] );
            Perm( list, k+1, m );
            Swap( list[k], list[i] );
        }
    }
}

template <class Type>
inline void Swap (Type &a, Type &b)
{
    Type temp = a;
    a = b;
    b = temp;
}
```

棋盘问题：

```
#define N 8
int board[N][N];
int tile = 1; // 骨牌编号

// 参数说明:
// tr, tc: 当前子棋盘左上角位置
// dr, dc: 当前特殊方格坐标（绝对坐标）
// size: 当前子棋盘规模（边长）
void chessBoard(int tr, int tc, int dr, int dc, int size) {
    if (size == 1) return;
    int t = tile++;
    int s = size / 2;
    // 左上子棋盘
    if (dr < tr + s && dc < tc + s) {
        chessBoard(tr, tc, dr, dc, s);
    } else {
        board[tr + s - 1][tc + s - 1] = t;
        chessBoard(tr, tc, tr + s - 1, tc + s - 1, s);
```

```
    }
    // 右上子棋盘
    if (dr < tr + s && dc >= tc + s) {
        chessBoard(tr, tc + s, dr, dc, s);
    } else {
        board[tr + s - 1][tc + s] = t;
        chessBoard(tr, tc + s, tr + s - 1, tc + s, s);
    }
    // 左下子棋盘
    if (dr >= tr + s && dc < tc + s) {
        chessBoard(tr + s, tc, dr, dc, s);
    } else {
        board[tr + s][tc + s - 1] = t;
        chessBoard(tr + s, tc, tr + s, tc + s - 1, s);
    }
    // 右下子棋盘
    if (dr >= tr + s && dc >= tc + s) {
        chessBoard(tr + s, tc + s, dr, dc, s);
    } else {
        board[tr + s][tc + s] = t;
        chessBoard(tr + s, tc + s, tr + s, tc + s, s);
    }
}
```

合并排序：

```
void MergeSort(Type a[], int left, int right)
{
    if (left < right) { // 至少有2个元素
        int i = (left + right) / 2; // 取中点
        mergeSort(a, left, i);
        mergeSort(a, i + 1, right);
        merge(a, b, left, i, right); // 合并到数组b
        copy(a, b, left, right);     // 复制回数组a
    }
}
void Merge(Type a[], Type b[], int l, int m, int r)
{
    int i = l, j = m + 1, k = l;
    while ((i <= m) && (j <= r)) {
        if (a[i] <= a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }
    if (i > m) {
        for (int q = j; q <= r; q++)
            b[k++] = a[q];
    }
    else {
        for (int q = i; q <= m; q++)
            b[k++] = a[q];
    }
```

```
}
```

快速排序：

```
template<class Type>
void QuickSort(Type a[], int p, int r)
{
    if (p < r) {
        int q = Partition(a, p, r);
        QuickSort(a, p, q - 1);    // 对左半段排序
        QuickSort(a, q + 1, r);    // 对右半段排序
    }
}

template<class Type>
int Partition(Type a[], int p, int r)
{
    int i = p, j = r + 1;
    Type x = a[p];
    // 将 < x 的元素交换到左边区域
    // 将 > x 的元素交换到右边区域
    while (true) {
        while (a[++i] < x && i < r);
        while (a[--j] > x);
        if (i >= j) break;
        Swap(a[i], a[j]);
    }
    a[p] = a[j];
    a[j] = x;
    return j;
}
```

寻找第k小元素：

```
template<class Type>
int RandomizedPartition(Type a[], int p, int r) {
    int i = Random(p, r);
    Swap(a[i], a[p]);
    return Partition(a, p, r);
}

template<class Type>
Type RandomizedSelect(Type a[], int p, int r, int k) {
    if (p == r) return a[p];
    int i = RandomizedPartition(a, p, r);
    int j = i - p + 1;
    if (k <= j)
        return RandomizedSelect(a, p, i, k);
    else
        return RandomizedSelect(a, i + 1, r, k - j);
}
```

```
template<class Type>
int Partition(Type a[], int p, int r)
{
    int i = p, j = r + 1;
    Type x = a[p];
    // 将 < x 的元素交换到左边区域
    // 将 > x 的元素交换到右边区域
    while (true) {
        while (a[++i] < x && i < r);
        while (a[--j] > x);
        if (i >= j) break;
        Swap(a[i], a[j]);
    }
    a[p] = a[j];
    a[j] = x;
    return j;
}
```

最近点对问题：

一维和二维（PDF挡上了）

循环赛问题：

```
void Table(int k, int **a)
{
    int n = 1;
    for (int i = 1; i <= k; i++)
        n *= 2;
    for (int i = 1; i <= n; i++)
        a[1][i] = i;
    int m = 1;
    for (int s = 1; s <= k; s++) {
        n /= 2;
        for (int t = 1; t <= n; t++) {
            for (int i = m + 1; i <= 2 * m; i++) {
                for (int j = m + 1; j <= 2 * m; j++) {
                    a[i][j + (t - 1) * m * 2] = a[i - m][j + (t - 1) * m * 2 -
m];
                    a[i][j + (t - 1) * m * 2 - m] = a[i - m][j + (t - 1) * m *
2];
                }
            }
        }
        m *= 2;
    }
}
```

动态规划

矩阵连乘

```
int LookupChain(int i, int j)
{
    if (m[i][j] > 0) return m[i][j];
    if (i == j) return 0;
    int u = LookupChain(i, i) + LookupChain(i+1, j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
        int t = LookupChain(i, k) + LookupChain(k+1, j) + p[i-1]*p[k]*p[j];
        if (t < u) { u = t; s[i][j] = k; }
    }
    m[i][j] = u;
    return u;
}
```

最长子序列

```
void LCSLength(int m, int n, char *x, char *y, int **c, int **b)
{
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0;
    for (i = 1; i <= n; i++) c[0][i] = 0;
    for (i = 1; i <= m; i++)
        for (j = 1; j <= n; j++) {
            if (x[i] == y[j]) {
                c[i][j] = c[i-1][j-1] + 1; b[i][j] = 1;
            } else if (c[i-1][j] >= c[i][j-1]) {
                c[i][j] = c[i-1][j]; b[i][j] = 2;
            } else {
                c[i][j] = c[i][j-1]; b[i][j] = 3;
            }
        }
}

void LCS(int i, int j, char *x, int **b)
{
    if (i == 0 || j == 0) return;
    if (b[i][j] == 1) { LCS(i-1, j-1, x, b); cout << x[i]; }
    else if (b[i][j] == 2) LCS(i-1, j, x, b);
    else LCS(i, j-1, x, b);
}
```

最大字段和

```
int MaxSum(int n, int *a)
{
    int sum = 0, b = 0;
    for (int i = 1; i <= n; i++)
    {
        if (b > 0)
            b += a[i];
        else
```

```
            b = a[i];
        if (b > sum)
            sum = b;
    }
    return sum;
}
```

最大子矩阵和

```
int MaxSum2(int m, int n, int **a)
{
    int sum = 0;
    int *b = new int[n+1];
    for (int i = 1; i <= m; i++) {
        for (int k = 1; k <= n; k++)
            b[k] = 0;
        for (int j = i; j <= m; j++) {
            for (int k = 1; k <= n; k++)
                b[k] += a[j][k];
            int max = MaxSum(n, b);
            if (max > sum)
                sum = max;
        }
    }
}
```

最大m子段和

```
int MaxSum(int m, int n, int *a)
{
    if (n < m || m < 1) return 0;
    int *b = new int[n+1]; b[0] = 0;
    int *c = new int[n+1]; c[1] = 0;
    for (int i = 1; i <= m; i++) {
        b[i] = b[i-1] + a[i];
        c[i-1] = b[i];
        int max = b[i];
        for (int j = i+1; j <= n-m+i; j++) {
            b[j] = b[j-1] > c[j-1] ? b[j-1] + a[j] : c[j-1] + a[j];
            c[j-1] = max;
            if (max < b[j])
                max = b[j];
        }
        c[i+n-m] = max;
    }
    int sum = 0;
    for (int j = m; j <= n; j++)
        if (sum < b[j])
            sum = b[j];
    return sum;
}
```

最优三角剖分

```
void MinWeightTriangulation(int n, Type **t, int **s)
{
    for (int i = 1; i <= n; i++) t[i][i] = 0;
    for (int r = 2; r <= n; r++) {  // 多边形节点个数为 r+1
        for (int i = 1; i <= n - r + 1; i++) {
            int j = i + r - 1;
            t[i][j] = t[i+1][j] + w(i-1, i, j);
            s[i][j] = i;
            for (int k = i+1; k < j; k++) { // j = i + r - 1
                int u = t[i][k] + t[k+1][j] + w(i-1, k, j);
                if (u < t[i][j]) {
                    t[i][j] = u;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

多边形游戏

```
void PolyMax(int n)
{
    int minf, maxf;
    for (int j = 2; j <= n; j++) {
        for (int i = 1; i <= n ; i++) {
            for(int s = 1; s < j; s++) {
                MinMax(n, i, s, j, minf, maxf);
                if (m[i][j][0] > minf)
                    m[i][j][0] = minf;
                if (m[i][j][1] < maxf)
                    m[i][j][1] = maxf;
            }
        }
    }
    int temp = m[1][n][1];
    for (int i = 2; i <= n; i++) {
        if (temp < m[i][n][1])
            temp = m[i][n][1];
    }
    return temp;
}
```

图像压缩

```
void Compress(int n, int p[], int s[], int l[], int b[])  // s[i] 第i个像素段长度
{
    int Lmax = 256, header = 11;
    s[0] = 0;
    for (int i = 1; i <= n; i++) {          // 像素点
```

```
        b[i] = length(p[i]);                // 像素点灰度值所占bit位
        int bmax = b[i];
        s[i] = s[i-1] + bmax;
        l[i] = 1;
        for (int j = 2; j <= i && j <= Lmax; j++) { // 像素段长度，j为该段最后一个节点
            if (bmax <= b[i-j+1]) bmax = b[i-j+1];
            if (s[i] > s[i-j] + j * bmax) {
                s[i] = s[i-j] + j * bmax;
                l[i] = j;
            }
        }
        s[i] += header;
    }
}
```

最优布线

```
// 电路布线最优值（动态规划主过程）
void MNS(int C[], int n, int **size)
{
    for (j = 0; j < C[1]; j++)
        size[1][j] = 0;
    for (j = C[1]; j < n; j++)
        size[1][j] = 1;

    for (i = 2; i < n; i++) {
        for (int j = 0; j < C[i]; j++)
            size[i][j] = size[i-1][j];
        for (int j = C[i]; j <= n; j++)
            size[i][j] = max(size[i-1][j], size[i-1][C[i]-1] + 1);
    }
    size[n][n] = max(size[n-1][n], size[n-1][C[n]-1] + 1);
}

// 路径回溯，恢复最优布线方案
void Traceback(int C[], int **size, int n, int Net[], int &m)
{
    int j = n;
    m = 0;
    for (int i = n; i > 1; i--) {
        if (size[i][j] != size[i-1][j]) {
            Net[m++] = i;
            j = C[i] + 1;
        }
    }
    if (j >= C[1])
        Net[m++] = 1;
}
```

0-1背包

```
template<class Type>
void knapsack(Type v[], int w[], int c, int n, Type **m)
{
    int jMax = min(w[n]-1, c);
    for (int j = 0; j <= jMax; j++)
        m[n][j] = 0;
    for (int j = w[n]; j <= c; j++)
        m[n][j] = v[n];

    for (int i = n-1; i > 1; i--) {
        jMax = min(w[i]-1, c);
        for (int j = 0; j <= jMax; j++)
            m[i][j] = m[i+1][j];
        for (int j = w[i]; j <= c; j++)
            m[i][j] = max(m[i+1][j], m[i+1][j-w[i]] + v[i]);
    }
    m[1][c] = m[2][c];
    if (c >= w[1])
        m[1][c] = max(m[1][c], m[2][c-w[1]] + v[1]);
}
```

最优二叉搜索树

```
void OptimalBinarySearchTree(int a[], int b[], int n, int **m, int **s, int **w)
{
    for (int i = 0; i <= n; i++) {
        w[i+1][i] = a[i];
        m[i+1][i] = 0;
    }

    for (int r = 0; r < n; r++) {
        for (int i = 1; i <= n - r; i++) {
            int j = i + r;
            w[i][j] = w[i][j-1] + a[j] + b[j];
            m[i][j] = m[i+1][j];
            s[i][j] = i;
            for (int k = i+1; k <= j; k++) {
                int t = m[i][k-1] + m[k+1][j];
                if (t < m[i][j]) {
                    m[i][j] = t;
                    s[i][j] = k;
                }
            }
            m[i][j] += w[i][j];
        }
    }
}
```